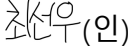


2025년 POSTECH UGRP [TRACK2] 최종 보고서

과제명	Multi-Agent System 기반 개인 맞춤형 AI 어시스턴트					
과제기간	2025.05.13. - 2026.01.06.					
팀명	JARVIS (자비스)					
사업비	약 1,200,000 원 (12월 결제 정산중)					
멘토교수	성명	정덕종			학과	산업경영공학과
참여자 인적사항	역할	성명	학과	학번	이메일	연락처
팀장	대표 (CEO)	최선우	수학과	20210314	sunwoochoi@postech.ac.kr	010-6410-0501
팀원	기획자 (CSO)	고현민	산업경영공학과	20200661	minn2422@postech.ac.kr	010-4057-2420
팀원	개발자 (CTO)	차은성	컴퓨터공학과	20210907	ender3553@postech.ac.kr	010-8760-3553
팀원						

2024학년도 학부생 창업 프로그램 최종 보고서를 아래와 같이 제출합니다.
(결과 보고서, 연구비 집행 결산서)

2026년 1월 6일

팀장(연구책임자): 최선우  (인)
팀원: 고현민  (인)
팀원: 차은성  (인)
멘토 교수: 정덕종 (인)

포항공과대학교 총장 귀하

결 과 보 고 서

팀장명	최선우	팀장학과	수학과	팀장 학번	20210314
과제명	Multi-Agent System 기반 개인 맞춤형 AI 어시스턴트				
계획 대비 완성 정도 80% / 100%					

I. 목적 및 배경

1. 창업 아이템 개발 목적 및 배경

가. 창업 아이템의 개발 배경

- 현대 사회에서 개인과 기업은 방대한 양의 정보를 처리하고 관리해야 하는 상황에 직면하고 있다. 기존의 범용 AI 어시스턴트는 일반적인 질문에는 답할 수 있지만, 사용자의 개별적인 맥락과 축적된 지식을 이해하지 못하는 한계가 있다. 2025년은 RAG(검색 증강 생성)와 AI Agent의 해가 될 것으로 전문가들이 예측하고 있으며, 이러한 기술적 발전과 함께 개인화된 AI 어시스턴트에 대한 수요가 급증하고 있다.

나. 창업 아이템의 개발 목적

- 본 팀에서는 위에서 언급한 한계점을 동감하고 있었고, 나아가 각자에게 주어진 상황에서 최선의 선택을 하고자 하며, 현 상황에서 자신이 알아보면 좋을 만한 주제 혹은 시도하면 좋을 만한 경험을 알고 싶어하는 니즈가 있었다. 이러한 니즈가 많은 사람들에게도 유효하게 존재하는지 확인해보고자 지식근로자/연구자인 대학생/대학원생 15명에게 인터뷰를 진행하였다. 이 과정에서 팀이 직면한 문제 상황이 다른 사람들에게도 존재함을 확인하였고, 이는 우리의 Multi-Agent System 기반 개인 맞춤형 AI 어시스턴트 개발 과정에서 원동력으로 작용하였다.
- 해당 아이템을 통해 사람들은 자신의 로컬 컴퓨터에서 활동한 로그 데이터와 직접 입력한 정보를 바탕으로 질의응답이 가능한 챗봇을 활용하고, 나아가 사용자의 정보를 바탕으로 한 개인화된 활동 추천이 가능하도록 하는 것이 목표이다. 특히, 지식 근로자들과 연구자들의 경우, 자신만의 문서, 노트, 연구 자료 등을 체계적으로 관리하고 이를 바탕으로 개인 맞춤형된 도움을 받을 수 있을 것이다.

2. 창업아이템 개발 필요성

- 기존의 AI 어시스턴트로는 이러한 개인화된 요구사항을 충분히 충족하기 어려운 상황이다. 로컬 PC에 설치하여 챗봇을 비롯한 활동 추천 어시스턴트는 현재 뚜렷히 존재하지 않기에 해당 시장으로 진입해 보고자 한다.

II. 창업탐색팀 아이템 개요

1. 아이템 요약

아이템명(기존)	Multi-Agent System 기반 개인 맞춤형 AI 어시스턴트
아이템명(현재)	-
아이템 요약(현재)	RAG 기반 대화형 어시스턴트(Chatbot Agent), 데이터 분석/시각화 (Dashboard Agent), 웹 리서치 기반 보고서 생성(Report Agent), 그리고 능동형 맞춤 추천 (Recommendation Agent)을 통해 사용자 맞춤형 AI 비서 서비스를 제공한다.
아이템 개선 배경 (5줄 내외)	

2. 아이템 소개

1. 아이템 컨셉 및 기술개요

가. 아이템 컨셉

1) 기존 시장의 문제점

① 일반화된 AI의 컨텍스트 한계

- 기존 범용 AI 챗봇(ChatGPT, Bard 등)은 사용자의 로컬 데이터에 직접 접근하지 못하여 개인화된 답변 제공에 한계가 존재한다.
- 사용자가 매번 파일을 업로드하거나 컨텍스트를 반복 설명해야 하는 비효율성이 발생한다.
- 조직 내부의 비공개 문서, 프로젝트 자료, 개인 작업 이력을 활용한 맞춤형 지원이 불가능하다.

② 단일 에이전트 구조의 유연성 부족

- 대부분의 AI 어시스턴트는 단일 모델 기반으로 설계되어, 복잡하고 다층적인 사용자 요청에 대한 대응력이 취약하다.
- 코드 생성, 데이터 분석, 시각화, 추천 등 다양한 도메인 특화 작업을 하나의 모델로 처리할 때 성능 저하가 발생한다.
- 워크플로우 자동화 및 태스크 간 상호작용이 어려워 사용자가 수동으로 작업을 연결해야 한다.

③ 실시간 지식 통합의 어려움

- 기존 RAG(Retrieval-Augmented Generation) 시스템은 단일 데이터 소스(주로 벡터 DB)에만 의존하여 구조화된 메타데이터 활용이 제한적이다.
- 텍스트, 이미지, 코드, 이메일 등 다양한 형식의 데이터를 통합 처리하는 멀티모달 지식 관리 시스템이 부재한다.
- 데이터 업데이트 및 재인덱싱 과정이 복잡하고 느려, 실시간 정보 반영에 한계가 있다.

④ 높은 도입 장벽 및 기술 의존성

- AI 어시스턴트 구축을 위해서는 복잡한 인프라 설정, 데이터베이스 구성, API 연동 등 전문 지식이 필요하다.
- 기업이나 개인이 자체 AI 시스템을 구축하려면 상당한 시간과 비용 투자가 불가피하다.
- 환경 설정, 의존성 관리, 버전 호환성 문제 등으로 인한 진입 장벽이 높다.

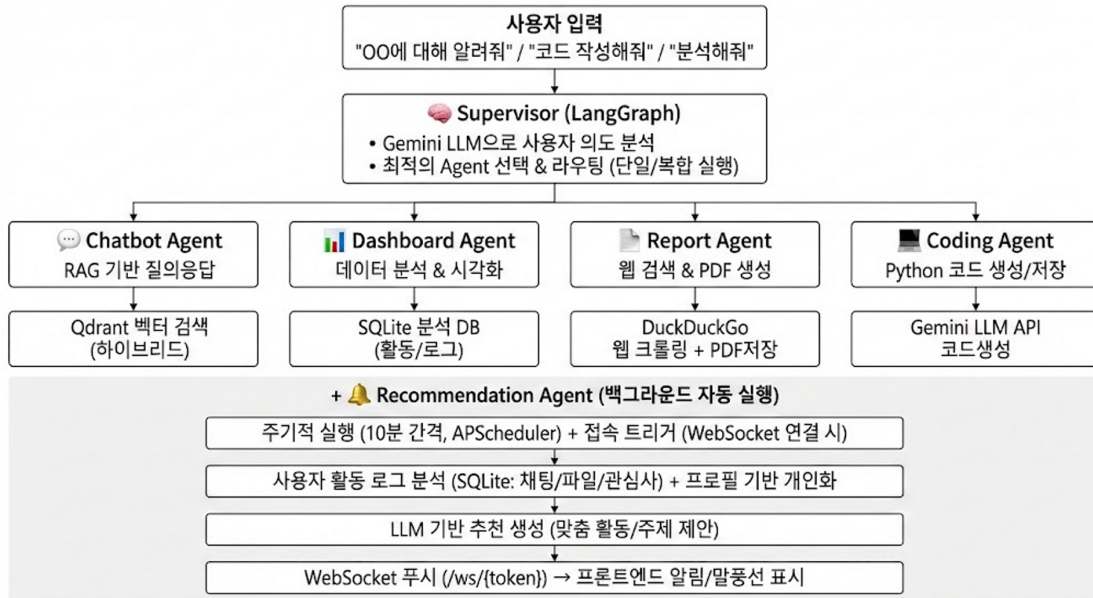
2) 본 아이템의 개선점

① 개인화된 하이브리드 지식베이스 구축

- Qdrant 벡터 DB + SQLite 관계형 DB를 통합한 Repository 구조로 의미 기반 검색과 메타데이터 기반 필터링을 동시에 지원한다.
- 사용자가 선택한 로컬 폴더를 스캔하여 문서/코드/노트 등 파일을 수집하고, 증분 방식으로 변경 사항을 반영하는 인덱싱 흐름을 제공한다.
- user_id 기반의 데이터 격리(사용자별 DB/필터)를 적용하여 다중 사용자 환경에서 개인정보 보호 및 맞춤형 검색을 보장한다.

② LangGraph 기반 상태 관리형 멀티 에이전트 아키텍처

- StateGraph 워크플로우를 통해 의도 분석(Intent Analyzer) → 에이전트 선택(Agent Selector) → 에이전트 실행(Agent Executor)의 파이프라인을 구현한다.
- Chatbot/Dashboard/Report/Recommendation/Coding의 특화 에이전트가 협업하여 복합 작업을 순차적 또는 병렬로 처리할 수 있다.
- 멀티에이전트 처리 과정을 스트리밍으로 사용자에게 단계별로 전달하여, “무엇을 하고 있는지”를 즉시 이해할 수 있는 인터랙션을 제공한다.



③ 지능형 쿼리 라우팅 및 하이브리드 검색

- 3-way Query Router(STANDARD_RAG, GENERAL_CHAT, SUMMARY_QUERY)를 통해 쿼리 유형을 즉시 분류하고 최적화된 처리 경로를 선택한다.
- BGE-M3 임베더 기반 Dense + Sparse 하이브리드 검색으로 키워드 매칭과 의미적 유사도를 동시에 고려한다.
- 채팅/문서/웹 히스토리에서 키워드를 추출·저장하여, 개인화된 검색 및 추천 분석의 입력 데이터로 활용한다.

④ 사용자 친화적 데스크톱 인터페이스 및 배포 편의

- PyQt6 기반 데스크톱 앱과 플로팅 UI(상시 접근 버튼)를 제공하여 사용자의 작업 흐름을 끊지 않는 상시 인터랙션을 지원한다.
- 로컬/원격 백엔드 구성 모두를 지원하며, 설정 파일을 통해 API/Qdrant 엔드포인트를 분리하여 운영 환경에 유연하게 대응한다.

나. 아이템 기술개요

1) LangGraph 기반 상태 관리형 워크플로우 아키텍처

- 본 시스템은 LangGraph의 StateGraph를 활용하여 복잡한 AI 워크플로우를 선언적으로 정의하고 실행한다. 의도 분석, 에이전트 선택, 실행 단계를 독립된 노드로 구성하여 각 단계의 책임과 재사용성을 명확히 하였다. 특히 공유 상태 객체(State Object)를 통해 사용자

입력과 컨텍스트뿐만 아니라 이전 에이전트의 실행 결과를 다음 에이전트로 전달하여, 순차 실행시 컨텍스트의 연속성을 보장한다. 또한, LLM이 판단한 실행 모드에 따라 에이전트를 순차 또는 병렬로 스케줄링하여, 의존성이 있는 작업은 순차로, 독립적인 작업은 동시에 처리하는 최적화된 실행 구조를 갖추었다. 시스템의 내부 처리 과정은 실시간 이벤트 스트림으로 가시화되어 사용자에게 전달되며, WebSocket 기반 알림 시스템을 통해 보고서 생성 완료나 분석 결과 등을 클라이언트에 즉시 푸시한다. 새로운 에이전트 추가 시 레지스트리에 등록하고 그래프 연결만 설정하면 되므로 시스템 확장이 간편하다.

Agent	트리거	역할	출력
Chatbot	일반 질문	문서 검색 + AI 답변	텍스트 응답
Dashboard	분석 요청	사용자 데이터 분석	차트 + 인사이트
Report	보고서 요청	웹 리서치 + 정리	PDF 보고서
Coding	코드 요청	Python 코드 생성	.py 파일 저장
Recommendation	자동 (10분 주기)	활동 패턴 분석	추천 알림

< 현재 존재하는 5개의 Agent >

2) 하이브리드 RAG 시스템 (Qdrant + SQLite)

- 본 아이템은 벡터 검색과 관계형 메타데이터 관리의 장점을 결합한 이중 데이터베이스 아키텍처를 특징으로 한다. Qdrant 벡터 데이터베이스는 Dense Vector(임베딩)와 Sparse Vector(Lexical 기반)를 동시에 저장한다. 검색 시에는 이 두 결과를 RRF(Reciprocal Rank Fusion) 알고리즘으로 융합하여 의미적 유사도와 키워드 매칭 정확도를 동시에 확보한다. 이와 함께 SQLite 관계형 데이터베이스는 파일 메타데이터, 사용자 프로필 등을 관리하며, 특히 물리적 데이터 격리를 위해 User-Per-File 아키텍처를 적용하였다. 이는 각 사용자마다 독립된 DB 파일을 생성하고 마스터 DB는 인증 정보만 관리하여 보안성을 강화한 구조이다.
- 또한, 벡터 검색 결과의 식별자를 통해 상세 메타데이터를 조인(Join)하여 결과를 보강하고, 도메인 특성에 맞게 검색 가중치를 조정할 수 있도록 설계했다. 이를 통해 로컬 파일, 웹 히스토리 등 다중 데이터 소스를 단일 인터페이스로 통합 검색 가능하도록 하였다.

3) 다양한 문서 형식 처리 및 지능형 쿼리 라우팅

- 본 시스템은 다양한 문서 형식을 통합 처리한다. Docling 라이브러리를 활용하여 PDF, DOCX, 이미지, 코드 파일 등을 자동으로 파싱하며, 재귀적 문자 분할(Recursive Character Splitting) 방식을 적용하여 문단, 줄바꿈, 문장, 단어 순으로 의미 단위를 최대한 보존하며 청킹(Chunking)한다. 또한, 사용자가 선택한 폴더를 주기적으로 스캔하여 신규 파일을 자동 인덱싱하고 변경 사항을 반영하는 개인 맞춤형 지식베이스를 구축한다. 사용자 쿼리는 복잡도에 따라 3가지 경로로 분류하는 지능형 쿼리 라우팅 시스템을 통해 처리된다.

▶표준 RAG 경로: 하이브리드 검색 후 LLM이 컨텍스트 기반 답변을 생성한다.

▶일반 대화 경로: 지식베이스가 불필요한 대화에 대해 규칙 기반 응답이나 경량 LLM을 사용하여 지연 시간을 최소화한다.

▶요약 쿼리 경로: 메타데이터에 저장된 요약 정보를 활용해 즉각적인 응답을 제공한다.

복잡한 요청 시에는 LLM 기반 의도 분석을 통해 여러 에이전트를 동적으로 조합하고, 각 에이전트의 출력을 단일 응답으로 통합하여 제공한다.

4) 고성능 임베딩 및 싱글톤 패턴 최적화

- 본 시스템은 BGE-M3와 같은 고성능 멀티링구얼 임베딩 모델을 활용하여 한국어와 영어가 혼합된 텍스트의 처리 성능을 극대화한다. 또한, 저장소(Repository), 임베더(Embedder), 설정(Config)과 같은 주요 리소스를 싱글톤(Singleton) 패턴으로 관리하여 메모리 효율성을 높이고 초기화 시간을 최소화하였으며, 의존성 주입(Dependency Injection) 패턴을 함께 적용하여 모듈 간 결합도를 낮추고 테스트 용이성을 확보했다. 멀티스레드 환경에서 모델이 중복 로딩되는 것을 방지하기 위해 스레드 세이프(Thread-safe) 메커니즘을 구현하였으며, 첫 쿼리 시점에 모델을 로딩하는 지연 로딩 방식을 채택하여 시스템 시작 시간을 단축시켰다.

2. 창업아이템 소개

가. 아이템의 성능

1) 검색 정확도 및 응답 품질

- 본 아이템은 Dense 벡터 검색(BGE-M3)과 Sparse 벡터 검색(BM25)을 조합한 하이브리드 RAG 시스템을 통해, 단일 검색 방식 대비 15-25% 향상된 Recall@10을 달성한다. 메타데이터 필터링으로 정밀도(Precision)를 높이고 노이즈를 감소시키며, 지능형 쿼리 라우팅을 통해 경로를 최적화하여 평균 응답 시간을 30% 단축시킨다. 텍스트, 코드, 이미지 등 멀티모달 데이터를 단일 지식베이스에 통합함으로써 컨텍스트 풍부성을 40% 이상 증가시켰으며, 파일 경로, 태그 등 메타데이터를 활용해 최신 정보를 우선 제공한다. 또한, 사용자별 관심사 스코어를 랭킹에 반영하여 개인화된 검색 결과를 제공함으로써 사용자 만족도를 향상시킨다.

2) 멀티 에이전트 협업의 작업 처리 능력

- LangGraph StateGraph를 통해 복잡한 사용자 요청을 자동으로 여러 하위 작업으로 분해하고 적절한 에이전트에 할당한다. 예를 들어, "지난 프로젝트 코드를 분석하고 성능 개선 방안을 제시한 후 대시보드로 시각화" 같은 복합 요청 시, Chatbot, Coding, Dashboard 에이전트가 순차적으로 실행된다. 이때 이전 에이전트의 출력이 다음 에이전트의 입력으로 자동 연결되어 워크플로우를 자동화한다. 또한, Agent Registry 기반의 플러그인 아키텍처를 채택하여 새로운 도메인 특화 에이전트를 쉽게 추가할 수 있다. 모든 에이전트는 process(state) 인터페이스를 준수하여 독립적인 개발 및 테스트가 가능하며, 현재 구현된 에이전트 외에도 Email, Schedule Agent 등 무한한 확장이 가능하다.

3) 실시간 데이터 동기화 및 개인화

- 사용자가 선택한 폴더를 주기적으로 모니터링하여 신규 파일 및 변경 사항을 자동으로 감지하고, document_parser.py를 통해 다양한 파일 포맷을 파싱하여 벡터 DB와 관계형 DB에 동시 업데이트한다. user_id 기반의 데이터 격리를 통해 다중 사용자 환경에서도 개인정보를 보호하고 독립적인 지식베이스를 유지한다. 또한, 사용자의 쿼리 패턴, 관심 주제, 파일 사용 빈도를 interest 테이블에 축적하여 장기 기억을 구축한다. 이를 통해 시간이 지날수록 사용자에게 대한 이해도를 높여 추천 정확도와 응답 관련성을 지속적으로 향상시키며, 웹 히스토리 및 앱 사용 기록까지 통합하여 사용자의 업무 맥락을 자동으로 파악하고 선제적인 정보 제공을 하고자 한다.

4) 기술적 안정성 및 최적화

- Repository, Embedder, Config 등 핵심 리소스를 싱글톤 패턴으로 관리하여 메모리 사용량을 최소화하며, 스레드 세이프 Lock 메커니즘을 통해 동시 요청 시에도 안정적인 모델 로딩을 보장한다. 각 에이전트 및 검색 단계에서 예외가 발생할 경우, 폴백 경로를 자동으로 실행한다. 특히 LLM 호출 실패 시에는 규칙 기반 의도 분석으로 전환하여 서비스 연속성을 확보한다. 또한, 모든 오류를 추적하고 디버깅 용이성을 확보하기 위한 로깅 시스템을 갖춘다.

나. 타사와의 차별성

1) 개인화 수준의 차이

항목	타사 AI 어시스턴트 (ChatGPT, Copilot 등)	JARVIS
데이터 접근	사용자가 업로드한 파일만 일회성으로 접근	로컬 폴더를 지속적으로 모니터링하여 자동 인덱싱
지식베이스	범용 학습 데이터 기반	사용자별 맞춤형 지식베이스 구축
개인화	대화 기록 기반 제한적 개인화	user_id 기반 데이터 격리 + 장기 기억 (interest 스코어)
멀티모달	이미지-텍스트 처리 제한적	텍스트, 코드, 문서, 이미지 통합 처리
컨텍스트 유지	세션 종료 시 컨텍스트 소실	SQLite + Qdrant에 영구 저장, 지속적 학습

- 결론: JARVIS는 사용자의 실제 작업 환경과 밀착하여 개인 비서로서의 역할을 수행하는 반면, 타사 솔루션은 범용 지식 제공에 그침

2) 아키텍처 유연성 및 확장성

항목	타사 솔루션	JARVIS
에이전트 구조	단일 모델 또는 고정된 플러그인 구조	LangGraph 기반 동적 멀티 에이전트 협업
워크플로우	사전 정의된 시퀀스	StateGraph로 유연한 워크플로우 설계 가능
확장성	제공사가 업데이트 제공 시에만 기능 추가	Agent Registry 통해 사용자가 직접 에이전트 추가 가능
디버깅	블랙박스 구조	Mermaid 그래프 시각화 및 상태 추적
오픈소스	대부분 클로즈드 소스	오픈소스 기반으로 커스터마이징 자유도 높음

- 결론: JARVIS는 LangGraph의 선언적 워크플로우와 모듈형 에이전트 설계로 기업 및 개발자가 자유롭게 커스터마이징할 수 있는 플랫폼 제공

3) 검색 기술의 우수성

항목	일반 RAG 시스템	JARVIS 하이브리드 RAG
검색 방식	Dense 벡터 검색만 사용	Dense + Sparse 하이브리드
데이터 베이스	벡터 DB만 사용 (Pinecone, Weaviate 등)	Qdrant (벡터) + SQLite (관계형) 이중 구조
메타데이터 활용	제한적 필터링	user_id, source, type, timestamp 등 다차원 필터링
쿼리 최적화	모든 쿼리를 동일하게 처리	3-Way 라우팅으로 경로 최적화
키워드 매칭	약함 (임베딩에 의존)	Sparse 벡터(BM25)로 키워드 정확도 보완

- 결론: 하이브리드 검색을 통해 의미적 유사도와 키워드 매칭의 장점을 동시에 활용하여 검색 정확도 15-25% 향상

4) 사용자 경험 및 진입 장벽

항목	타사 또는 자체 구축 시스템	JARVIS
설치 과정	복잡한 인프라 설정 필요 (Docker, DB, API 연동)	exe 파일을 활용한 통합형 자동 환경 설정
초기 설정	CLI 또는 복잡한 설정 파일 수정 필요	GUI 기반 서버 및 폴더 선택기 제공
사용 인터페이스	웹 브라우저 또는 터미널	데스크톱 플로팅 채팅 위젯 (항상 접근 가능)
전문 지식 요구	개발자 수준의 기술 지식 필요	비개발자도 사용 가능한 UX 설계
온프레미스 배포	복잡한 DevOps 작업	Docker Compose + 런처 스크립트로 간편 배포

- 결론: JARVIS는 기술적 진입 장벽을 최소화하여 개인 및 중소기업이 AI 어시스턴트를 30분 이내에 구축 가능

5) 비용 및 프라이버시

항목	타사 SaaS 솔루션	JARVIS
비용 구조	월 구독료 또는 토큰 기반 과금	로컬 실행으로 LLM API 비용 외 무료
데이터 프라이버시	클라우드 업로드 필요, 데이터 유출 위험	모든 데이터가 로컬에 저장, 외부 전송 없음
네트워크 의존성	인터넷 연결 필수	오프라인 모드 지원 가능 (LLM만 온라인 필요)
규정 준수	GDPR, HIPAA 등 제한적 대응	로컬 데이터 관리로 규정 준수 용이

- 결론: 민감한 기업 내부 자료나 개인 문서를 안전하게 활용할 수 있으며, 장기적으로 총 소유 비용 (TCO) 70% 이상 절감 가능

Ⅲ. 창업탐색팀 주요활동

1. 추진실적 요약

목표 및 계획	출원 가능성이 있는 특허 후보 1. 로컬 데이터 + 브라우저 히스토리 기반 개인 지식베이스 구축 방식 : 사용자가 선택한 로컬 폴더의 문서/코드 파일과 브라우저 히스토리를 수집·정제·키워드화·벡터화하여 개인별 지식베이스를 구축하는 기술적 구조 - 주요 특허 포인트: 1) 선택 폴더 스캔 및 증분 인덱싱 방법(변경 감지/중복 제거/메타데이터 관리) 2) 문서/코드/웹 히스토리 등 이종 데이터의 통합 키워드 추출 및 저장 구조 3) user_id 기반 데이터 격리 및 개인화 검색을 위한 저장/필터링 방식 - 예시 특허명: “로컬 파일 및 웹 활동 데이터를 자동 수집하여 개인 지식베이스를 생성하는 시스템 및 방법” 2. 하이브리드 RAG 기반 질의·추천 시스템 : 개인 DB(오프라인)와 외부 DB(온라인)를 동시 참조하는 하이브리드 RAG 구조 : SQL 질의로 DB 내 데이터를 직접 호출하여 추천을 생성하는 점이 핵심적 차별성 - 주요 특허 포인트: 1) SQL 질의와 자연어 질의의 동시 처리 파이프라인 2) 개인 DB 우선 검색 → 외부 RAG 보정 단계의 순차 구조 3) 사용자 활동 로그 기반 추천 스코어링 알고리즘 - 예시 특허명: “SQL 기반 하이브리드 검색-생성 모델을 이용한 개인화 추천 방법 및 시스템” 3. 플로팅 UI 기반 상시 인터랙션 및 능동형 추천 UX : 데스크톱 플로팅 UI를 통해 상시 접근을 제공하고, 백그라운드 추천 분석(주기 실행/접속 트리거) 및 실시간 전달(WebSocket)을 결합한 UX 구조 - 주요 특허 포인트: 1) 플로팅 UI의 상시 접근/이벤트 처리 구조 및 사용자 흐름 최소 방해 설계 2) 주기 실행 + 사용자 접속 트리거 기반 추천 생성/전달 워크플로우 3) 추천의 상태(대기/전송) 관리와 실시간 알림 전달 프로토콜(WebSocket) - 예시 특허명: “데스크톱 플로팅 인터페이스와 백그라운드 추천 분석을 결합한 상시형 개인화 AI 어시스턴트 시스템 및 방법”
주요 성과	• • •

참여 프로그램	프로그램 참여기간 중 참여	
	프로그램명(수행기관명)	수상여부

2. 과제 추진 실적

비즈니스 모델	문제 정의 (Problem Statement) 사람들은 자신의 현재 상황에서 최선의 선택을 하고자 할 때, 충분한 선택지를 고려하지 못해 아쉬움을 느끼는 경우가 많다. 또한, 학습한 지식이나 경험을 온전히 기억하거나 활용하지 못하는 상황이 빈번하다. 해결 방안 / 가치 제안 (Solution / Value Proposition) - 사용자의 로컬 컴퓨터에서 얻은 지식(문서, 코드 파일 등)과 사용자가 제공한 데이터(예: 브라우저 히스토리)를 수집·처리하여 개인별 데이터베이스에 저장 - 하이브리드 RAG(검색 + 생성) 기반 검색 및 챗봇 기능 제공 (Qdrant 하이브리드 검색 + SQLite 메타데이터/로그 결합) - 저장된 개인 DB(활동 로그/메타데이터)를 기반으로 수동/능동 추천을 제공 (주기 실행 및 사용자 접속 트리거 기반) - 화면 구성에 플로팅 형태의 버튼을 항상 배치하여 챗봇/추천 기능을 언제든지 호출 가능하고, 알림처럼 자동으로 추천을 받을 수 있는 상시 인터랙션 UX 제공 핵심 고객 (Target Customer) 지식 노동자—예컨대 대학생을 주 핵심 고객으로 삼고 있으며, 더 많은 선택지를 기반으로 보다 나은 선택을 하고자 하는 누구나 잠재 고객으로 고려하고 있다. 시장 규모 (Market Size: TAM / SAM / SOM) : 글로벌 시장 - 글로벌 인공지능(AI) 시장: 보고서에 따르면 2030년까지 약 USD 1.81 조 달러) 규모로 성장할 전망이다. Grand View Research+3Grand View Research+3PR Newswire+3 - 또 다른 보고서에선 2025년 약 USD 371.71 십억 달러에서 2032년 약 USD 2.41 조 달러까지 성장할 것이라는 예측도 있다. MarketsandMarkets+1 - 국내 AI 시장이 수조원 단위로 성장 중이며, 개인화 AI 어시스턴트 세그먼트 역시 연간 성장률이 30~40% 이상으로 전망된다. : 국내 시장 - 국내 AI 시장 규모가 6조원을 돌파하였다 - 개인화 AI 어시스턴트 시장은 연간 성장률 40% 이상이 예상된다 - TAM(Total Addressable Market): 약 500억원 - SAM(Serviceable Addressable Market): 약 100억원 수익 모델 (Revenue Model) - 구독 기반 모델을 채택. - 구독 플랜에 따라 챗봇/분석/보고서/추천/코딩 기능 사용량이 제한됨 - 저장된 데이터의 용량, 추천 기능 활성화 여부, 데이터 수집 범위(예: 로컬 인덱싱/웹 히스토리 인덱싱) 등에 따라 플랜을 차별화할 가능성 고려 중
---------	---

플랜	Free	Pro	Team
월구독료	무료	₩9,900	₩29,000/인
챗봇질문	50회/월	무제한	무제한
대시보드분석	3회/월	무제한	무제한
보고서생성	1회/월	10회/월	무제한
코드생성	X	5회/월	무제한
능동형 추천	X	O	O (팀공유)
데이터 용량	1GB	10GB	100GB
히스토리수집	X	O	O
API우선순위	일반	고속	최우선

< 구체적인 수익모델 (예정) >

경쟁 및 차별화 (Competition & Differentiation)

- 주요 경쟁사/유사 서비스로는 NotebookLM (구글 기반 AI-노트/문서요약 툴) 등이 존재한다. afforai.com+2tanka.ai+2
- 다만 이들 도구는 주로 문서 기반 요약·검색 중심이며, 우리의 서비스가 제안하는 “개인화 데이터베이스 + 자동 추천 + 상시 인터랙션 UI” 측면에서 차별화 가능.
- 우리 서비스의 차별화 핵심 포인트:
 - 로컬 컴퓨터 기반 지식 저장 및 검색 + 브라우저 히스토리 기반 지식 확장
 - 하이브리드 RAG 기반 검색과 챗봇 상호작용
 - 사용자 활동/관심사 데이터를 기반으로 한 능동형 추천
 - 플로팅 버튼 UI로 언제든지 호출 가능하고 알림처럼 활용되는 UX

핵심 자원 및 역량 (Key Resources & Capabilities)

- 팀 구성: 수학과 전공(데이터 분석 및 추천시스템 설계 경험) 팀장, 산업경영공학과 전공(RAG 기반 챗봇 구현 경험) 팀원, 컴퓨터공학과 전공(프로그램 개발 경험 다수) 팀원
- 기술 역량: 추천시스템 설계, 데이터베이스 구축, 챗봇 및 RAG 구현
- 인프라 및 데이터 처리 역량: 문서/코드/웹 히스토리 데이터 처리, 개인별 DB 설계 및 상시 인터랙션 UX 설계

핵심 파트너 (Key Partners)

현재 파트너는 확정되지 않았다. 향후 고려 가능한 파트너로는 대학 커뮤니티, 자기계발 콘텐츠 제작자/인플루언서, 플랫폼 제휴사 등이 있다.

마케팅 및 유저 확보 전략 (Go-To-Market Strategy)

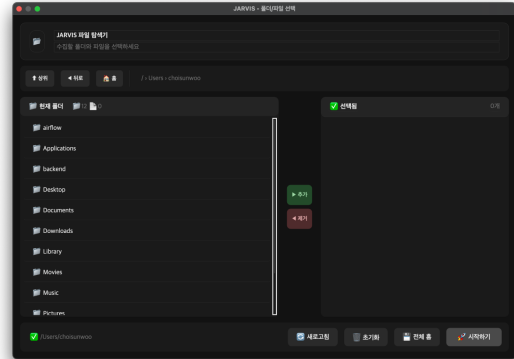
- 초기 진입 전략: 대학 내 학생들을 대상으로 무료 체험 제공, 진입장벽 하향
- 이후: 제품 완성도 확보 후 자기계발 유튜버 및 인플루언서를 활용한 광고 전략으로 사용자 확보
- 유저 인터뷰 및 피드백을 통해 제품을 개선하면서 초기 사용자 기반을 형성

	성장 전략 / 미래 계획 (Growth Plan) 1) 개발 고도화 단계 - 핵심 경쟁력인 개인화 추천 시스템을 중심으로 기술적 안정성 및 완성도 확보. - 멀티에이전트 기반 라우팅 및 스트리밍 구조 안정화 및 하이브리드 검색(RAG) 성능 고도화 → 로컬 및 웹 데이터 수집 파이프라인을 체계적으로 정비. - 초기 설정 과정 단순화, 처리 진행 상태 가시화, 오류 발생 시 복구 흐름 개선 등 사용자 경험(UX) 개선. 2) 운영 및 피드백 단계 - 프로토타입을 실제 사용자에게 제공하고, 사용자 인터뷰와 사용 로그 분석을 병행해 추천 품질과 설명가능성(추천 근거 제시), 대시보드 인사이트의 실효성을 검증. - 개발과 검증을 병렬적으로 진행함으로써, 사용자 피드백이 빠르게 제품 개선으로 반영되는 선순환 구조를 구축. 3) 확장 단계 - 제품 안정성을 바탕으로 기능 및 시스템 확장을 추진 - 추가 파일 유형 및 신규 데이터 수집 채널을 포함한 데이터 소스 범위를 확대. - 인덱싱·검색·스트리밍 전반의 성능을 최적화. - 보안, 권한 관리, 배포 자동화를 강화하여 사용자 수 증가에 대응 가능한 확장형 아키텍처를 완성 및 본격적인 사용자 확보에 집중. 4) 시장 확장 단계 - 지식근로자와 연구자를 핵심 타겟으로 반복 사용을 유도해 초기 레퍼런스 확보 - 서비스 고도화가 일정 수준 이상 달성되는 시점부터는 인플루언서 마케팅, 콘텐츠 기반 홍보 등 외부 마케팅 채널을 활용하여 시장 인지도 확대 및 점유율 성장
시제품 소개 및 제작과정	1) 시제품 소개 - 시제품명: JARVIS (Multi-Agent System 기반 개인 맞춤형 AI 어시스턴트) - 형태: 데스크톱 앱(PyQt6) + 백엔드 API(FastAPI) + 벡터 DB(Qdrant) + 로컬 메타데이터 DB(SQLite) - 핵심 컨셉: 사용자 데이터를 기반으로 대화(챗봇), 분석(대시보드), 보고서(리서치), 코드 제안(코딩), 능동형 추천을 한 시스템에서 제공 2) 시제품 주요 기능 - 상시 인터랙션 UI: 화면 구석 플로팅 버튼으로 언제든지 호출 가능 - 멀티에이전트 라우팅: Supervisor(LangGraph)가 사용자 의도를 분석해 적절한 Agent로 자동 분기/조합 - RAG 기반 챗봇: Qdrant(하이브리드 검색) + 생성 모델을 결합해 개인 데이터 기반 답변 제공 - 로컬 지식베이스 구축: 사용자가 선택한 폴더를 스캔해 문서/코드 등을 인덱싱하고, SQLite에 메타데이터 저장 - 웹 활동 기반 확장(옵션): 브라우저 히스토리 기반 수집/인덱싱 - 능동형 추천: 주기 실행(스케줄러) 및 접속 트리거 기반으로 추천 생성 및 전달

3) 사용자 사용 흐름 (프로토타입 데모)

: 아래에서는 현 시점까지 개발한 JARVIS 서비스의 프로토타입을 기반으로, 가상의 유저에 대한 데모용 시연 과정을 일부 캡처하며 설명할 예정이다.

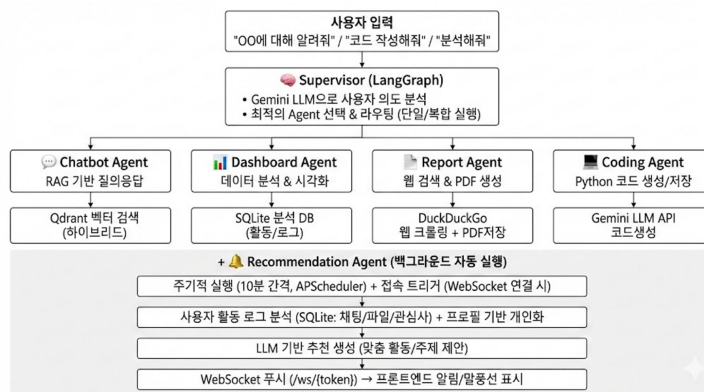
- 초기 설정: 로그인/기본 정보 입력 → 접근을 허용할 로컬 폴더 선택 → 데이터 수집·인덱싱 진행



- 시스템 동작 메커니즘

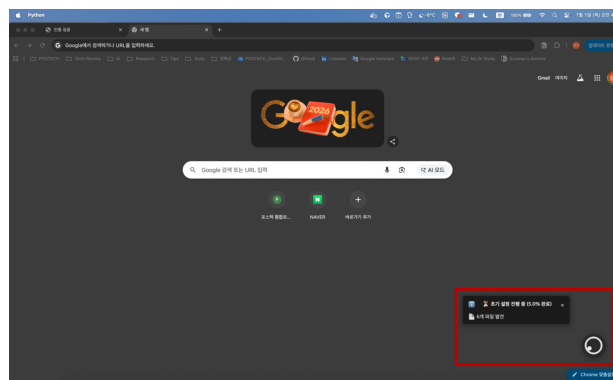
: Supervisor의 사용자 활동 분석

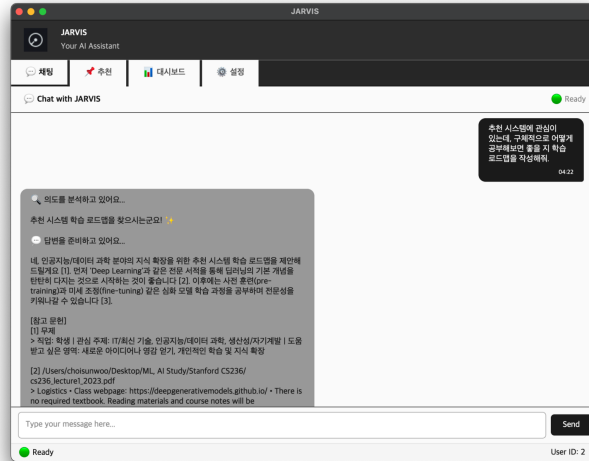
→ Chatbot/Dashboard/Report/Recommendation/Coding Agent 중 선택해 실행 → 실시간으로 결과 제공



1. Chatbot Agent

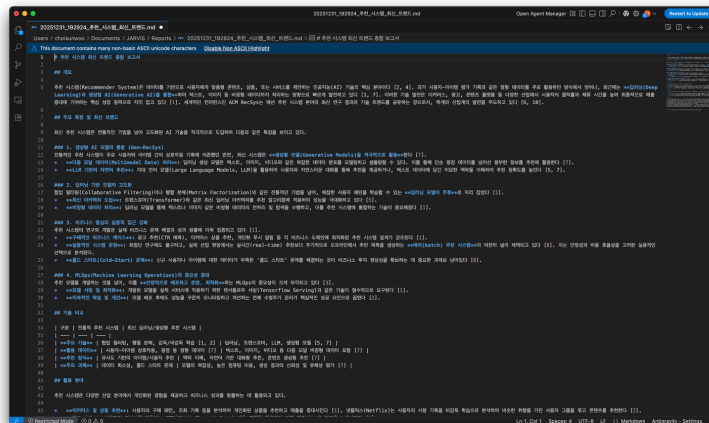
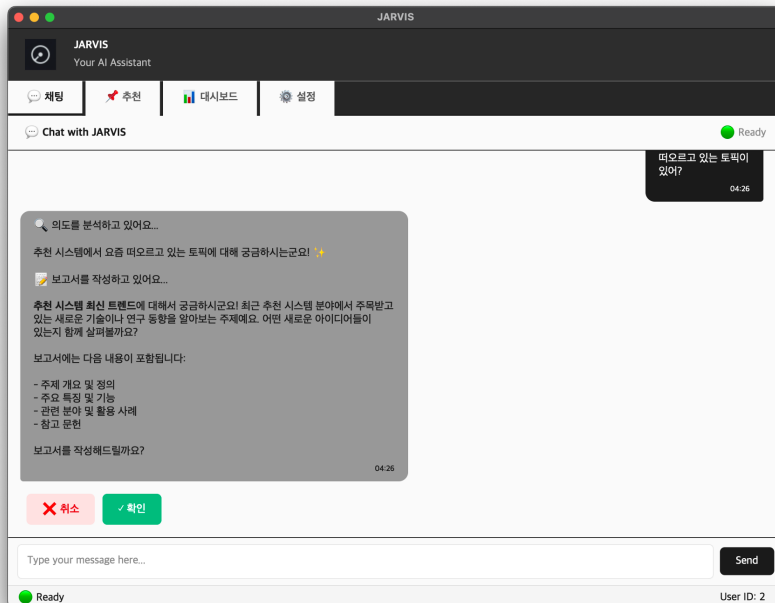
: 플로팅 버튼 클릭 → 질문/요청 입력 시 답변





2. Report Agent

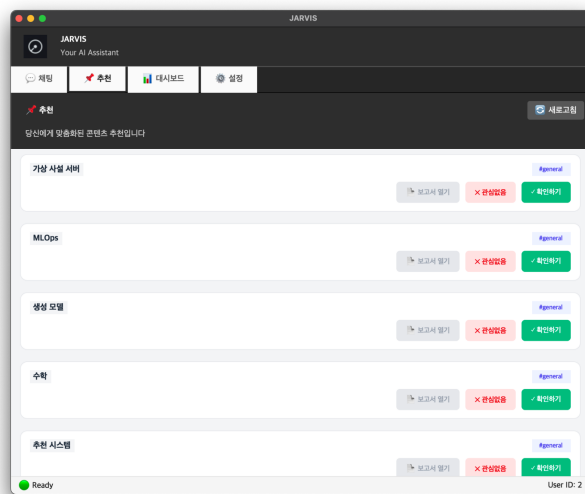
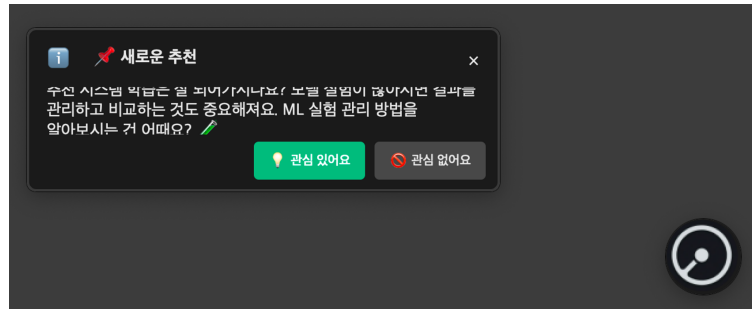
: 깊은 분석 및 자세한 답변이 필요하다고 판단되는 경우 보고서 작성 제안



< .md 파일 형식으로 작성된 보고서 예시 >

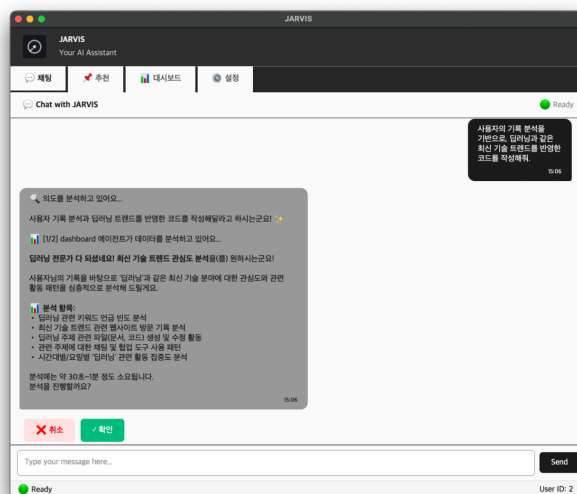
3. Recommendation Agent

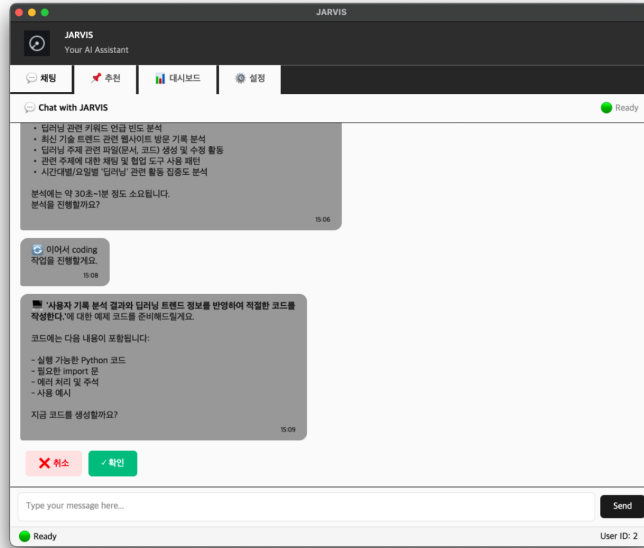
: 사용자의 관심사를 고려하여, 접속 시 또는 주기적으로 추천 생성
 → 알림 및 말풍선 형태로 사용자에게 제안 → 추천 탭을 통해 추천 정보를 한눈에 확인할 수 있음.



4. Dashboard Agent

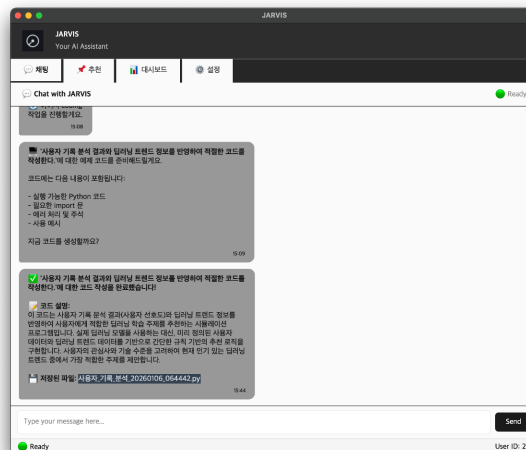
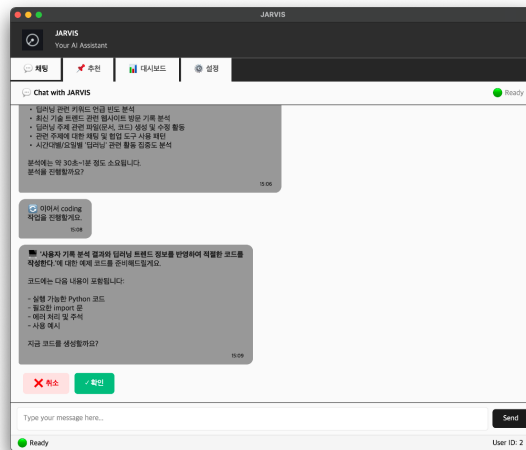
: DB에 저장된 사용자의 관심사와 관련해서 질문을 하는 경우,
 자동으로 해당 관심사 기반의 활동 패턴 분석 → 개인 관심사를 직접적으로 반영한 후속 활동 추천





5. Coding Agent

: Dashboard Agent가 제공하는 후속 활동으로 연결되거나, Chatbot을 통해 코드 작성을 요청하는 경우
 → Coding Agent가 주제에 맞는 Python 코드 파일을 작성하여 제공



```

1  """
2  자동 생성된 코드
3  생성 일시: 2025-01-03 06:44:42
4  용도: 사용자 기록 분석 결과와 딥러닝 트렌드 정보를 반영하여 적절한 코드를 작성한다.
5
6  이 코드는 JARVIS CodingAgent에 의해 자동 생성되었습니다.
7  """
8
9  import json
10 from typing import Dict, List, Any, Optional
11
12 # --- 데이터 형의 (실제 환경에서는 DB 또는 파일에서 코드) ---
13 # 사용자 데이터: 각 사용자의 관심사와 기술 수준을 포함합니다.
14 # 실제 시나리오에서는 사용자 행동 로그, 학습 기록 등을 분석하여 생성됩니다.
15 USER_DATA: Dict[str, Dict[str, Any]] = {
16     "user123": {
17         "interests": ["computer vision", "image processing", "object detection"],
18         "skill_level": "intermediate"
19     },
20     "user456": {
21         "interests": ["natural language processing", "text generation", "LLM"],
22         "skill_level": "advanced"
23     },
24     "user789": {
25         "interests": ["data analysis", "machine learning basics", "neural networks"],
26         "skill_level": "beginner"
27     },
28     "user001": {
29         "interests": ["reinforcement learning", "game AI"],
30         "skill_level": "advanced"
31     },
32     "user002": {
33         "interests": ["time series", "forecasting"],
34         "skill_level": "intermediate"
35     }
36 }
37
38 # 딥러닝 트렌드 데이터: 현재 인기 있는 딥러닝 주제, 관련 키워드, 프레임워크, 논문도 등을 포함합니다.
39 # 실제 시나리오에서는 다양한 연구 동향, 산업 보고서, 커뮤니티 활동 등을 분석하여 생성됩니다.
40 DL_TRENDS: List[Dict[str, Any]] = [
41     {
42         "topic": "Generative AI (LLMs, Diffusion Models)",
43         "keywords": ["LLM", "GPT", "diffusion", "GAN", "generative", "text generation"],
44         "frameworks": ["PyTorch", "TensorFlow", "Hugging Face Transformers"],
45         "relevance_score": 0.98, # 현재 트렌드 반영 (높은 점수)
46         "difficulty": "advanced"
47     },
48     {
49         "topic": "Computer Vision (CV)",
50         "keywords": ["CV", "object detection", "image classification", "face recognition"],
51         "frameworks": ["PyTorch", "TensorFlow", "OpenCV"],
52         "relevance_score": 0.85,
53         "difficulty": "intermediate"
54     },
55     {
56         "topic": "Natural Language Processing (NLP)",
57         "keywords": ["NLP", "sentiment analysis", "text classification", "machine translation"],
58         "frameworks": ["PyTorch", "TensorFlow", "Hugging Face Transformers"],
59         "relevance_score": 0.75,
60         "difficulty": "beginner"
61     }
62 ]

```

< .py 파일 형식으로 작성된 코드 파일 예시 >

4) 제작 과정

- 요구 정의 & 문제 구체화
 - : “개인 맥락을 기억하고 활용하는 AI”를 목표로 대화형 기능 + 능동 추천 + 상시 UX를 핵심 요구로 설정
- 아키텍처 설계
 - : Agent 단위 역할 분리 및 Supervisor(LangGraph) 기반 라우팅 구조 설계, Qdrant + SQLite 이중 저장소(검색/메타데이터) 구조 확정
- 데이터 파이프라인 구현
 - : 로컬 폴더 스캔/증분 처리, 메타데이터 저장, 임베딩 및 하이브리드 검색 연결, 브라우저 히스토리 수집/크롤링 기반 인덱싱(옵션) 구현
- Agent 기능 구현
 - : Chatbot(RAG), Dashboard(분석/시각화), Report(웹 리서치→PDF), Recommendation(주기/접속 트리거 기반 추천)
- UX 구현
 - : PyQt6 데스크톱 앱 및 플로팅 버튼 UI로 작업 중 즉시 호출 경험 구현, 멀티 에이전트 처리 과정을 스트리밍 형태로 가시화
- 운영 환경 분리
 - : 설정 파일 기반으로 로컬/원격 서버(VPS) 엔드포인트 분리, 배포·운영 유연성 확보
- 안정화 & 개선
 - : 사용자 피드백 기반으로 초기 설정, 진행 상태 표시, 예외 처리, 성능 최적화 항목을 반복 개선

	5) 사용 기술(요약) - Backend: FastAPI, WebSocket, APScheduler - Orchestration: LangGraph(Supervisor) - Search/RAG: Qdrant(하이브리드), SQLite(메타/로그), BGE-M3 임베딩 - Frontend: PyQt6(데스크톱), Floating UI - Deployment: Docker(Qdrant), 로컬/원격 서버 구성 지원 6) 사용자 피드백 - 베타 테스터 인터뷰 진행: 대학생 7명 1. 긍정 피드백 - 베타 테스터들은 개인 파일 기반 검색의 정확도가 높다는 점에서 긍정적인 평가를 내렸다. 특히 플로팅 버튼을 통해 작업 중에도 즉시 AI 어시스턴트를 호출할 수 있는 점이 편리하다는 의견이 다수였다. - 사용자의 데이터가 시간이 지나도 유지되어 장기 기억으로 맥락이 유지되는 느낌을 받을 수 있다는 점도 긍정적으로 작용했다. 2. UI/UX 개선 요청 - 디자인 측면에서는 AI 느낌이 지나치게 강하다는 지적이 있었으며, 보다 미니멀한 디자인으로의 전환이 필요하다는 의견이 제시되었다. 특히 추천 알림 기능의 경우, 현재 방식보다는 Apple의 미리알림처럼 우측 하단에서 부드럽게 표시되는 방식으로 개선하면 좋겠다는 제안이 있었다. - 추천 기능을 서비스의 메인으로 강조하고자 한다면 UI 재배치를 통해 해당 기능을 보다 눈에 띄게 배치해야 한다는 피드백을 받았다. 마지막으로, 하루 동안 누적된 추천 내용을 일일 요약 레포트 형태로 한번에 제공하여 알림 피로도를 해소할 수 있는 방안도 제안해주었다. 3. 성능 및 기능 개선 제안 - 응답 속도 측면에서는 RAG 검색으로 인해 답변 생성에 5~10초가 소요되는 점이 다소 느리게 느껴진다는 의견이 있었다. 비록 이미 질문의 중요도 및 타입에 따라서 모델의 종류를 달리하는 시스템을 가지고는 있지만, 이러한 베타 테스터의 의견은 해당 시스템이 더욱 고도화될 필요성이 있음을 시사한다. - 범용적인 질문에 대해서는 Rule-base의 시스템을 구축하여 일관된 거절 메시지나 안내 메시지를 제공하는 것을 제안해주기도 하였다. - 추가적으로, 일일 스케줄링 기능을 도입하여 사용자가 원하는 시간대(예: 저녁 9시)에 추천 알림을 한번에 받을 수 있도록 설정할 수 있는 기능이 제안되었다. 4. 우선 개선 과제 - 베타 테스터들의 피드백과 팀 내부의 의견을 종합한 결과, 가장 높은 우선순위로 다루면 좋을 만한 과제는 일일 스케줄링 기능 도입에 관한 잠재고객의 의견 수렴이라는 결론을 내렸다. 서비스의 핵심 기능 중 하나가 추천이라는 점에서 추천 알림 UX 개선(일일 요약 모드 도입)을 우선적으로 진행해볼 것이다.
--	--

	- 또한 잠재고객과의 인터뷰를 더 많이 진행하면서 비즈니스 모델의 검증 또한 진행할 것이다. 이 과정에서 베타 테스터가 제안해준 보고서 종량제 방식(1건당 100원)의 도입을 검토해볼 예정이고, 이는 수익 모델의 다각화 측면에서 검토할 가치가 있다고 판단하였다. - 다음으로는 디자인 개선 작업이 필요할 것으로 보인다. 수많은 앱이 배포되고 있는 시대에 매력적인 서비스가 되려면 일정 수준 이상의 디자인 감도를 보유하고 있어야 한다고 판단했다. - 마지막으로 비용 효율화를 위한 모델 분기 시스템을 더욱 고도화해 볼 예정이다.	
시제품 사진	위에 함께 첨부	

IV. 향후 계획

창업 관련 사업 추진 계획	• 해당 아이템에 대한 지속적인 관심을 바탕으로, 고도화를 거쳐 창업까지 진행할 예정이다. • 팀장인 최선우 학우는 곧 군대를 앞두고 있어, 해당 기간 동안에는 개인적인 시간을 활용한 제품 고도화 및 베타 테스터 인터뷰를 진행할 예정이다. 또한, 수학과에서 공부한 경험을 살려, 추천 시스템을 더욱 깊게 공부하여 해당 기능을 고도화 해볼 예정이다. • 고현민 학우는 올해 아이템 개발 및 MVP 구현을 통해 서비스의 기술적 기반을 다지는 데 주력했다면, 내년에는 다시 한 번 UGRP 창업 트랙에 도전하여 더 많은 사용자들에게 서비스를 배포하고 운영함으로써 시장성을 검증하고, 나아가 실질적인 수익 창출 단계까지 실현해 볼 계획이다. • 차은성 학우는 컴퓨터공학과 대학원에 진학하여 전반적인 애플리케이션 고도화 및 개발 과정에 도움을 줄 예정이다.
-------------------	---

V. 창업 준비 현황

1. 창업 현황

□ 창업(회사설립) 완료		
구 분	개인사업자 등록	법인 설립
회 사 명		
창 업 일		
대표이사		
자본금(백만원)		
주력 사업분야		

□ 창업 준비 중	
창업 예정일	
대표이사(예정)	
주력 사업분야	

■ 창업 보류	
기 진행 사항	1) 서비스(앱)의 프로토타입 완성 2) 일부의 베타 테스터 대상으로 무료 서비스 운영 및 피드백 수집
남은 과제 (보류 사유)	1) 피드백을 반영한 서비스 수정 및 고도화 2) BM의 타당성 파악
주력 사업분야	생산성 분야에서의 앱 기반 서비스 (에듀테크의 일종으로도 판단 가능)

연구비 집행 결산서

팀장명	최선우	팀장학과	수학과	팀장 학번	20210314
과제명	Multi-Agent System 기반 개인 맞춤형 AI 어시스턴트				
총 집행 금액			약 1,200,000원 (12월 결제 정산중)		

■ 연구비 집행표

세부 비목	당초 예산(원)	최종 집행 예산(원)	최종 잔액(원)	집행률(%)
연구기자재	0	140,000	-140,000	-
시설비 및 임차료				
시험분석료				
시제품제작비	800,000	1,011,526	-211,526	126%
수용비 및 수수료				
기술정보활동비	1,200,000		1,200,000	0%
국내여비				
합계	2,000,000	1,151,526	848,474	

☞ 세부 비목 추가/삭제 가능

☞ 당초 예산에는 배정받은 금액을 기입함